

# Ch04 CSS

## 投影片

:point\_right: [nlh Slide- CSS](#)

- Basic CSS
- CSS Selectors
  - Simple; Combinator; Pseudo-class; Pseudo-element; Attribute selector
- Layout and Box model
- Style and Effects
- Responsive Design and Advanced Topics

:point\_right: [W3school CSS](#)

:point\_right: [nlh GitHub www-js](#)

- **CSS demo**
- Bootstrap demo
- Javascript demo

## 4.1 CSS 介紹

CSS (Cascading Style Sheets) 是用來控制網頁樣式和佈局的技術。它允許開發者定義網頁元素的外觀，如顏色、字體、間距、佈局等。CSS 可以與 HTML 一起使用，通過選擇器將樣式應用於特定的 HTML 元素。簡單來說，HTML 用於結構化內容，而 CSS 則負責讓這些內容更具視覺吸引力和可讀性。

CSS 主要由三部分組成：

1. **選擇器**：指定要應用樣式的 HTML 元素。
2. **屬性**：定義樣式的具體行為（例如顏色、大小等）。
3. **值**：屬性的具體設置（例如顏色值、像素大小等）。

例如，下面的 CSS 代碼將一個段落文字的顏色設為紅色：

```
p {  
  color: red;  
}
```

## 4.2 主要選擇器

:point\_right: [Demo: CSS Selectors](#)

:point\_right: [W3Schools - CSS Selectors](#)

CSS 提供多種選擇器，讓開發者能夠根據不同條件精確地選擇和設置 HTML 元素的樣式。這裡介紹五個主要的選擇器。

---

**範例程式 (自我介紹頁面)**

在接下來的練習中，我們可以使用這個簡單的「自我介紹」HTML 結構作為練習的基礎。你可以將這段代碼複製到你的編譯器中運行，然後嘗試套用不同的選擇器。

```
<!DOCTYPE html>
<html lang="zh-Hant">
<head>
  <meta charset="UTF-8">
  <title>我的自我介紹</title>
  <style>
    /* 基礎樣式 */
    body {
      font-family: Arial, sans-serif;
      line-height: 1.6;
      margin: 20px;
    }
    .container {
      max-width: 600px;
      margin: auto;
      border: 1px solid #ddd;
      padding: 20px;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1 id="main-title">大家好，我是 [你的名字]</h1>

    <p class="intro">我是一名正在學習網頁前端開發的學生，對 <strong>CSS 選擇器
</strong> 非常感興趣。</p>

    <h2>我的興趣</h2>
    <ul>
      <li>閱讀技術部落格</li>
      <li class="active">撰寫網頁專案</li>
      <li>參加技術研討會</li>
    </ul>

    <h2>聯絡我</h2>
    <p>你可以透過以下方式找到我：</p>
    <a href="https://github.com" target="_blank" title="我的
GitHub">GitHub 首頁</a>
    <a href="mailto:example@fcu.edu.tw">發送郵件</a>
  </div>
</body>
</html>
```

---

### 4.2.1 簡單選擇器

簡單選擇器根據元素的名稱、ID、類名來選擇元素。常見的簡單選擇器有三種類型：

- **元素選擇器**：根據 HTML 元素的標籤名稱選擇元素。

| 符號 | 名稱             | 範例                                      | 說明                                                    |
|----|----------------|-----------------------------------------|-------------------------------------------------------|
| #  | id selector    | #ssn { background-color: yellow; }      | <div id="ssn"> B123456789</div><br>全文應只有一個地方 id 為 ssn |
| .  | class selector | .note { background-color: yellow; }     | <div class="note"> I am a note </div>                 |
|    |                | p.note { background-color: yellow; }    | <p class="note"> This is a note under p </p>          |
| ,  | group selector | p, h1, h2 { background-color: yellow; } | 標記為 p, h1, h2 的背景設為黃色                                 |

```
p {  
  color: blue;  
}
```

這樣的選擇器會選擇所有 `<p>` 標籤的元素，並將它們的文字顏色設為藍色。

- **ID 選擇器**：根據元素的 `id` 屬性選擇單一元素。 `id` 必須是唯一的。

```
#header {  
  background-color: gray;  
}
```

這樣的選擇器會選擇 ID 為 `header` 的元素，並設置它的背景顏色為灰色。

- **類選擇器**：根據元素的 `class` 屬性選擇一組元素。可以應用於多個元素。

```
.button {  
  padding: 10px;  
  background-color: green;  
}
```

這樣的選擇器會選擇所有具有 `button` 類名的元素，並設置內邊距和背景顏色。

**小練習：簡單選擇器** 請基於「自我介紹範例程式」，嘗試完成以下練習：

1. 使用 **ID 選擇器** 將主標題 ( `#main-title` ) 的文字顏色改為綠色。

2. 使用 類控制選擇器 將簡介段落 ( `.intro` ) 的文字大小改為 `1.2rem` 並加粗。
3. 使用 元素選擇器 將所有 `<h2>` 的下方補上 10px 的間距 ( `margin-bottom: 10px` )。

#### 4.2.2 組合選擇器

組合選擇器根據元素之間的結構關係來選擇元素。這些選擇器可以是直接或間接關係的元素。

| 符號   | 名稱                              | 範例                                                    | 說明          |
|------|---------------------------------|-------------------------------------------------------|-------------|
| (空格) | <b>descendant</b>               | <code>div p { background-color: yellow; }</code>      | div 下的所有 p  |
| >    | <b>child</b>                    | <code>div &gt; p { background-color: yellow; }</code> | div 下第一層的 p |
| +    | <b>Adjcent sibling Selector</b> | <code>div + p { background-color: yellow; }</code>    | div 同層鄰居的 p |
| ~    | <b>General sibling Selector</b> | <code>div ~ p { background-color: yellow; }</code>    | div 同層的 p   |

- **\*\*子元素選擇器 ( > )\*\***：選擇直接嵌套在某個元素內的子元素。

```
div > p {  
  color: red;  
}
```

這樣的選擇器會選擇所有直接位於 `div` 元素內的 `<p>` 元素。

- **\*\*後代元素選擇器 (空格)\*\***：選擇所有嵌套在某個元素內的後代元素。

```
div p {  
  font-size: 16px;  
}
```

這樣的選擇器會選擇所有位於 `div` 元素內的 `<p>` 元素。

- **\*\*相鄰兄弟選擇器 ( + )\*\***：選擇緊接在某個元素後面的相鄰兄弟元素。

```
h1 + p {  
  margin-top: 20px;  
}
```

這樣的選擇器會選擇位於 `<h1>` 元素後面的第一個 `<p>` 元素。

- **\*\*一般兄弟選擇器 ( ~ )\*\***：選擇所有緊跟在某個元素後的兄弟元素。

```
h1 ~ p {
  color: green;
}
```

這樣的選擇器會選擇所有位於 `<h1>` 元素後的 `<p>` 元素。

小練習：組合選擇器 請基於「自我介紹範例程式」，嘗試完成以下練習：

1. 使用 子元素選擇器 (`>`) 僅將 `.container` 下的首層 `<h2>` 顏色改為藍色。
2. 使用 後代選擇器 將所有 `<ul>` 中的 `<li>` 加上圓點符號 (`list-style-type: circle`)。
3. 使用 相鄰兄弟選擇器 (`+`) 讓緊接在 `<h2>` 後方的段落文字加粗。

小練習：請同學練習看看，使用以下這些屬性選擇器會對元素產生什麼樣的效果？

| Selector                         | Example                           | Example description      |
|----------------------------------|-----------------------------------|--------------------------|
| <code>[attribute]</code>         | <code>[target]</code>             | 有特定屬性的元素                 |
| <code>[attribute=value]</code>   | <code>[target="_blank"]</code>    | 屬性有特定值的元素                |
| <code>[attribute~=value]</code>  | <code>[title~="flower"]</code>    | 屬性值包含特定字的元素 (空白分隔)       |
| <code>[attribute =value]</code>  | <code>[lang "en"]</code>          | 屬性值以特定字開頭， - 分隔的元素 (或沒有) |
| <code>[attribute^=value]</code>  | <code>a[href^="https"]</code>     | 屬性值以特定字開頭的元素             |
| <code>[attribute\$=value]</code> | <code>a[href\$=".pdf"]</code>     | 屬性值以特定字結尾的元素             |
| <code>[attribute*=value]</code>  | <code>a[href*="w3schools"]</code> | 屬性值包含特定字的元素              |

小練習：屬性選擇器 請基於「自我介紹範例程式」，嘗試完成以下練習：

1. 使用 `[target="_blank"]` 選擇器，將外連連結的後方加上一個小圖標或標記（例如 (EXT)）。
2. 使用 `[title]` 選擇器，將所有擁有 `title` 屬性的元素加上下虛線。
3. 使用 `[href^="mailto:"]` 選擇器，將郵件連結的文字設為紅色。

#### 4.2.3 偽類選擇器

偽類選擇器根據元素的狀態或位置選擇元素，常用於處理用戶交互或元素的狀態。

- `** :hover **`：當元素被鼠標懸停時觸發。

```
a:hover {
  color: red;
}
```

這樣的選擇器會讓 `<a>` 元素在鼠標懸停時變成紅色。

- **\*\* :first-child \*\***：選擇某元素是其父元素的第一個子元素。

```
p:first-child {  
  font-weight: bold;  
}
```

這樣的選擇器會讓每個父元素的第一個 `<p>` 元素加粗。

- **\*\* :nth-child() \*\***：選擇某元素是其父元素中的第幾個子元素。

```
li:nth-child(2) {  
  color: blue;  
}
```

這樣的選擇器會選擇父元素中的第二個 `<li>` 元素並將其文字顏色設為藍色。

- **\*\* :active \*\***：當元素被激活（點擊）時觸發。

```
button:active {  
  transform: scale(0.98);  
}
```

- **\*\* :focus \*\***：當元素獲得焦點（如點擊輸入框）時觸發。

```
input:focus {  
  border-color: blue;  
  outline: none;  
}
```

- **\*\* :checked \*\***：當單選框 (radio) 或複選框 (checkbox) 被勾選時觸發。

```
input:checked + label {  
  font-weight: bold;  
  color: red;  
}
```

- **\*\* :not(selector) \*\***：選擇不符合括號內條件的元素。

```
li:not(.special) {  
  color: gray;  
}
```

**小練習：偽類選擇器** 請基於「自我介紹範例程式」，嘗試完成以下練習：

1. 使用 **:hover** 讓滑鼠移到聯絡連結 ( `<a>` ) 上時，背景色變為淡黃色。

2. 使用 `:nth-child(even)` 將興趣列表中的偶數項目背景改為灰色。
3. 使用 `:active` 讓點擊連結時有稍微縮小的動畫效果 ( `transform: scale(0.95)` )。

---

[!TIP] 🚀 旅遊網站對照：互動狀態 (States)

在旅遊網站中，我們常用偽類來增強按鈕的點擊感。以下是傳統 CSS 與 Tailwind 的實現對照。

**傳統 CSS:**

```
.btn-book {
  background-color: #2563eb; /* blue-600 */
  transition: all 0.3s;
}
.btn-book:hover {
  background-color: #1d4ed8; /* blue-700 */
}
.btn-book:active {
  transform: scale(0.95);
}
```

**Tailwind CSS:**

```
<button class="bg-blue-600 hover:bg-blue-700 active:scale-95 transition-all">
  立即預訂
</button>
```

---

#### 4.2.4 偽元素選擇器

偽元素選擇器允許開發者選擇元素的一部分並設置樣式。常見的偽元素包括：

- `** ::before` \*\*：在元素內容之前插入內容。

```
p::before {
  content: "Note: ";
  font-weight: bold;
}
```

這樣的選擇器會在每個 `<p>` 元素的內容前插入 "Note: "。

- `** ::after` \*\*：在元素內容之後插入內容。

```
p::after {
  content: " (end)";
}
```

這樣的選擇器會在每個 `<p>` 元素的內容後插入 "(end)"。

- `** ::first-letter **`：選擇元素的第一個字母。

```
p::first-letter {  
  font-size: 2em;  
  color: red;  
}
```

這樣的選擇器會使每個 `<p>` 元素的第一個字母變大並顯示為紅色。

- `** ::selection **`：選擇用戶正在選取的文字部分。

```
::selection {  
  background: yellow;  
  color: black;  
}
```

- `** ::placeholder **`：選擇輸入元素的提示文字 (placeholder)。

```
input::placeholder {  
  color: #ccc;  
  font-style: italic;  
}
```

- `** ::first-line **`：選擇元素內容的第一行。

```
p::first-line {  
  font-variant: small-caps;  
  color: darkblue;  
}
```

**小練習：偽元素選擇器** 請基於「自我介紹範例程式」，嘗試完成以下練習：

1. 使用 `::selection` 讓使用者選取網頁文字時，背景變為紫色，文字變為白色。
2. 使用 `::first-line` 將自我介紹段落的第一行設為粗體。
3. 使用 `::after` 在每個 `<h2>` 的標題後方加上一個符號  ( `content: "  "` )。

#### 4.2.5 屬性選擇器

屬性選擇器讓你能根據元素的屬性來選擇元素。

| Selector                 | Example               | Example description |
|--------------------------|-----------------------|---------------------|
| <code>[attribute]</code> | <code>[target]</code> | 有特定屬性的元素            |



|                      |                      |                          |
|----------------------|----------------------|--------------------------|
| [attribute=value]    | [target="_blank"]    | 屬性有特定值的元素                |
| [attribute~=value]   | [title~= "flower"]   | 屬性值包含特定字的元素 (空白分隔)       |
| [attribute =value]   | [lang = "en"]        | 屬性值以特定字開頭， - 分隔的元素 (或沒有) |
| [attribute^=value]   | a[href^="https"]     | 屬性值以特定字開頭的元素             |
| [attribute\$=value]  | a[href\$=".pdf"]     | 屬性值以特定字結尾的元素             |
| [attribute*="value"] | a[href*="w3schools"] | 屬性值包含特定字的元素              |

[!TIP] 屬性選擇器差異詳解：這些選擇器在處理「部分匹配」時非常有用，特別是當屬性包含多個值或特定格式時：

- **[attr~="val"]** (包含單字)：選擇屬性值中包含 `val` 這個完整單字 (以空格分隔) 的元素。
  - 範例：`<div class="btn primary">` 會被 `[class~="btn"]` 匹配。
- **[attr|= "val"]** (特定開頭)：選擇屬性值是 `val` 或以 `val-` 開頭的元素 (常用於語言標籤)。
  - 範例：`<p lang="en-US">` 會被 `[lang|= "en"]` 匹配。
- **[attr^="val"]** (開頭)：選擇屬性值以 `val` 開頭的元素。
  - 範例：`<a href="https://example.com">` 會被 `[href^="https"]` 匹配。
- **[attr\$="val"]** (結尾)：選擇屬性值以 `val` 結尾的元素。
  - 範例：`` 會被 `[src$=".jpg"]` 匹配。
- **[attr\*="val"]** (包含字串)：選擇屬性值中包含 `val` 任何片段的元素。
  - 範例：`<a href="news-2024.html">` 會被 `[href*="2024"]` 匹配。
- **\*\* [attr] \*\***：選擇擁有指定屬性的元素。

```
input[type="text"] {
  border: 1px solid blue;
}
```

這樣的選擇器會選擇所有 `type` 屬性為 `text` 的 `<input>` 元素並為其添加邊框。

- **\*\* [attr="value"] \*\***：選擇屬性值等於指定值的元素。

```
a[href="https://example.com"] {
  color: green;
}
```

這樣的選擇器會選擇所有 `href` 屬性為 `https://example.com` 的 `<a>` 元素，並設置其顏色為綠色。

## 4.1 測驗題

1. 哪個 CSS 選擇器會選擇所有 `<div>` 標籤的元素？
  - A. `.div`
  - B. `#div`
  - C. `div`
  - D. `*`
2. 如何選擇 `id="header"` 的元素？
  - A. `header`
  - B. `.header`
  - C. `#header`
  - D. `*header`
3. `a:hover` 是哪種類型的選擇器？
  - A. 簡單選擇器
  - B. 組合選擇器
  - C. 偽類選擇器
  - D. 偽元素選擇器
4. 以下哪一個選擇器會選擇父元素中的第三個 `<p>` 元素？
  - A. `p:nth-child(3)`
  - B. `p:nth-of-type(3)`
  - C. `p:first-child`
  - D. `p:last-child`
5. 什麼 CSS 選擇器可以根據元素的 `type` 屬性選擇所有 `text` 類型的輸入框？
  - A. `[type="text"]`
  - B. `input[type="text"]`
  - C. `.text`
  - D. `input`

## 4.2 測驗題答案：

► 點擊展開看解答

---

## 4.1 練習題

### 練習一

1. 使用 `::before` 偽元素在每個 `<h2>` 標題前添加 "Chapter: "。
2. 使用 `:nth-child()` 選擇器將每個父元素中的奇數項 `<li>` 文字顏色設為紅色。
3. 使用屬性選擇器，選擇所有 `type="checkbox"` 的 `<input>` 元素並設置它們的背景顏色為黃色。

練習二 4. 課表應用：使用 `:first-child` 選擇器，將課表表格中每一行的第一個儲存格（時段欄位）背景顏色設為 `#f2f2f2`，並將字體加粗。

5. 課表應用：使用 \*\*後代選擇器 (Descendant Selector)\*\*，將課表中所有包含在 `<td>` 內的超連結 ( `<a>` ) 顏色設為橘色 ( `orange` )，並移除下標線 ( `text-decoration: none` )。

### 4.3 尺寸單位

:point\_right: [Demo: CSS Units](#)

:point\_right: [W3Schools - CSS Units](#)

單位定義與特性表

| 單位  | 類型 | 基準點 (Reference)                         | 特點                   |
|-----|----|-----------------------------------------|----------------------|
| px  | 絕對 | 螢幕像素 (Pixel)                            | 固定大小，最精確但不具備彈性。      |
| em  | 相對 | 父元素 的字體大小                               | 會產生「複利效應」，層級愈深計算愈複雜。 |
| rem | 相對 | 根元素 ( <code>&lt;html&gt;</code> ) 的字體大小 | 全域統一基準，目前網頁設計的主流建議。  |
| vw  | 相對 | 視窗寬度 (Viewport Width)                   | 隨瀏覽器視窗大小即時縮放。        |

#### px (Pixels)

- 本質：它是最直覺的單位。如果你設 `20px`，無論在什麼裝置上它就是 20 個像素點。
- 缺點：不利於\*\*無障礙設計 (Accessibility)\*\*。如果使用者在瀏覽器設定「大字體」，使用 `px` 的網頁文字不會隨之放大。

#### em (Relative to Parent)

- 邏輯：如果父元素是 `16px`，子元素設 `2em` 就會變成 `32px`。
- 風險：如果在嵌套結構中使用（例如多層列表），每一層都會乘以父層的大小，導致字體縮放失控。

#### rem (Root em)

- 邏輯：`rem` 的 `r` 代表 **Root**。它永遠只參考 `<html>` 標籤的大小（預設通常是 `16px`）。
- 優點：它解決了 `em` 的複利問題，同時又保有彈性。當使用者調整瀏覽器字級時，整個網頁會成比例縮放。
- 公式：`$$Target\ Px / Root\ Px = rem$$`

(例如：`$24px / 16px = 1.5rem$`)

#### vw (Viewport Width)

- 邏輯：`1vw` 等於視窗寬度的 `**1%**`。
- 應用：常用於製作「大標題」或「滿版佈局」，確保在手機或超大螢幕上，元素比例都能維持一致。

2. 邊框與細微偏移：使用 `** px **`。因為 1 像素的邊框不需要隨字體縮放。

[!TIP] 🚀 旅遊網站對照：尺寸單位 (Units)

在旅遊網站的 Hero Section 中，我們常使用 `rem` 來確保標題與間距在不同裝置上的比例協調。

傳統 CSS:

```
.hero-section {  
  padding: 3rem;          /* 相當於 48px */  
  text-align: center;  
}  
.hero-title {  
  font-size: 2.25rem;    /* 相當於 36px */  
  margin-bottom: 1rem;   /* 相當於 16px */  
}
```

#### Tailwind CSS:

```
<section class="p-12 text-center">  
  <h1 class="text-4xl mb-4">探索未知的世界</h1>  
</section>
```

(註：Tailwind 的 `p-12` 代表  $12 \times 0.25\text{rem} = 3\text{rem}$ )

#### 測驗題

1. 如果使用者在瀏覽器設定中調大字體大小，哪種單位會隨之縮放？

- A. px
- B. rem
- C. cm
- D. pt

2. em 單位是相對於誰的字體大小？

- A. 根元素 ( `<html>` )
- B. 視窗高度
- C. 父元素
- D. 螢幕像素

3. 在瀏覽器預設 (16px) 下，2.5rem 等於多少像素？

- A. 32px
- B. 40px
- C. 20px
- D. 48px

4. 若想製作一個寬度佔滿視窗一半的元素，應使用哪個單位？

- A. 50vw
- B. 50vh
- C. 50rem
- D. 50px

5. 以下關於 px 的敘述何者正確？

- A. 它是相對單位，具備高度彈性
- B. 它是製作響應式網頁（RWD）最推薦的文字單位
- C. 它能精確控制大小，但不利於無障礙設計
- D. 它的基準點是父元素的寬度

#### 4.3 測驗題答案：

► 點擊展開看解答

#### 練習題

「如果 `<html>` 是 `16px`，父元素是 `2rem`，子元素是 `1.5em`，請問子元素實際是多少像素？」

答案解析：

- 父元素 =  $16 \times 2 = 32\text{px}$
- 子元素 =  $32 \times 1.5 = 48\text{px}$

## 4.4 排版與盒模型

:point\_right: [Demo: Box Model](#)

:point\_right: [W3Schools - CSS Box Model](#)

在 CSS 中，**佈局 (Layout)** 和 **盒模型 (Box Model)** 是設計網頁結構和樣式的基礎概念。理解這些概念有助於更有效地控制網頁元素的顯示和排列。

### 4.4.1 盒模型

盒模型是 CSS 中用來描述元素如何在頁面上呈現的框架結構。每個元素都被視為一個矩形框，它由以下四個部分組成：

- **Content (內容區域)**：元素的實際內容，如文字或圖片，這部分的大小由元素的 `width` 和 `height` 設定決定。
- **Padding (內邊距)**：內容區域與邊框之間的間距，內邊距不影響元素的總寬度或總高度，但會增加元素的可視區域。這部分的大小可用 `padding` 屬性設定。
- **Border (邊框)**：包圍內容和內邊距的邊框，邊框的寬度、樣式和顏色可以通過 `border` 屬性進行設定。
- **Margin (外邊距)**：元素外部與其他元素之間的空白區域。外邊距不會影響元素的寬度或高度，但會影響元素之間的間距。這部分的大小可以用 `margin` 屬性設定。

**Box Model 範例：**

```
div {
  width: 200px;
  height: 100px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

在這個例子中，div 的實際顯示區域會是：

- 實際內容區域寬度：200px
- 內邊距：20px
- 邊框：5px
- 外邊距：10px

這樣，div 的總寬度會是：200px（內容）+ 20px（左內邊距）+ 20px（右內邊距）+ 5px（左邊框）+ 5px（右邊框）= 250px。

---

#### 4.4.2 彈性盒佈局

:point\_right: [Demo: Flexbox](#)

:point\_right: [W3Schools - CSS Flexbox](#)

Flexbox 是一種一維佈局工具，專門用來處理元素在一條線上的對齊與空間分配。它是目前網頁開發中最主流的佈局方式。

**Flexbox 核心屬性：**

- **容器屬性 (Container)：**
  - `display: flex;`：啟用彈性盒模型。
  - `flex-direction`：控制主軸方向 ( row 橫向, column 縱向)。
  - `justify-content`：主軸對齊 ( center , space-between , space-around )。
  - `align-items`：交錯軸對齊 ( center , start , end )。
  - `flex-wrap`：是否換行 ( wrap , nowrap )。
- **項目屬性 (Items)：**
  - `flex-grow`：成長比例。
  - `order`：排列順序。

**Flexbox 實作範例：**

```
<style>
  .flex-container {
    display: flex;
    flex-direction: row;
    justify-content: center;
    flex-wrap: wrap;
    align-items: start;
    background-color: #f0f0f0;
    padding: 10px;
  }
  .item {
    border: 1px solid red;
    width: 50px;
    height: 50px;
    background-color: antiquewhite;
    margin: 10px;
  }
```

```
    text-align: center;
    line-height: 50px;
  }
</style>

<div class="flex-container">
  <div class="item">1</div>
  <div class="item" style="height:80px">2</div>
  <div class="item">3</div>
</div>
```

#### [!TIP] 🚀 旅遊網站對照：佈局與間距 (Flex/Grid)

旅遊網站展示多個目的地時，使用 Flexbox 或 Grid 可以輕鬆達成。

##### 傳統 CSS (Flexbox):

```
.gallery {
  display: flex;
  justify-content: space-between;
  gap: 2rem; /* gap-8 */
}
```

##### Tailwind CSS (Grid):

```
<div class="grid grid-cols-1 md:grid-cols-3 gap-8">
  <div>日本 東京</div>
  <div>法國 巴黎</div>
  <div>冰島</div>
</div>
```

### 4.4.3 box-sizing 屬性介紹

在 CSS 中，`box-sizing` 屬性控制著如何計算元素的寬度和高度。它決定了元素的總寬度和高度是否包含內邊距（padding）和邊框（border）的尺寸。`box-sizing` 有兩個主要的值：

#### content-box (預設值)

這是 CSS 的預設值。當 `box-sizing` 設置為 `content-box` 時，`width` 和 `height` 只會計算元素的內容區域，不包括內邊距和邊框的大小。也就是說，元素的總寬度和總高度是由內容區域的寬高，加上內邊距和邊框來確定的。

例子：

```
div {
  width: 200px;
```

```
padding: 20px;
border: 5px solid black;
box-sizing: content-box;
}
```

在這個例子中：

- 內容區域寬度是 200px
- 內邊距是 20px
- 邊框是 5px

元素的總寬度將是： $200\text{px}$ （內容）+  $20\text{px}$ （左內邊距）+  $20\text{px}$ （右內邊距）+  $5\text{px}$ （左邊框）+  $5\text{px}$ （右邊框）=  $250\text{px}$ 。

---

### border-box

當 `box-sizing` 設置為 `border-box` 時，`width` 和 `height` 會包括元素的內容區域、內邊距和邊框的大小。也就是說，元素的總寬度和總高度將等於設置的 `width` 和 `height`，並且內邊距和邊框的尺寸會從內容區域的寬高中扣除。

例子：

```
div {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: border-box;
}
```

在這個例子中：

- 總寬度是 200px（包含了內邊距和邊框）
- 內邊距和邊框會在內容區域內扣除，所以內容區域的實際寬度會減少。

元素的內容區域寬度會是： $200\text{px} - 20\text{px}$ （左內邊距）-  $20\text{px}$ （右內邊距）-  $5\text{px}$ （左邊框）-  $5\text{px}$ （右邊框）=  $150\text{px}$ 。

---

### 使用場景

- **content-box**：當你希望確保元素的內容區域具有固定的寬度和高度，並且不關心內邊距和邊框的尺寸對元素總尺寸的影響時，使用 `content-box`。
- **border-box**：當你希望元素的總寬度或總高度固定，並且想要內邊距和邊框包含在內時，使用 `border-box`。這對於設計響應式佈局特別有用，因為它能確保元素在調整尺寸時保持一致。

---

### 通常的做法

為了避免每次都手動調整元素的寬度和高度，許多開發者會將 `box-sizing` 設置為 `border-box`，並將此設定應用於所有元素。這樣能確保所有元素的總寬度和高度不會被內邊距和邊框的大小所影響，簡化布



局計算。

```
* {  
  box-sizing: border-box;  
}
```

這行代碼會將所有元素的 `box-sizing` 設置為 `border-box`，這樣每個元素的寬度和高度將包含內邊距和邊框的尺寸。

---

### 小結

- `content-box`：寬度和高度只計算內容區域，不包括內邊距和邊框。
- `border-box`：寬度和高度計算包含內邊距和邊框，內容區域的寬度和高度會減少。

使用 `border-box` 能使佈局更加簡單和直觀，這也是許多現代 CSS 框架（如 Bootstrap）推薦的做法。

---

### 測驗題

1. 哪個 CSS 屬性用來設定元素的內邊距？
  - A. `border`
  - B. `padding`
  - C. `margin`
  - D. `width`
2. 若一個元素有 `width: 100px`，`padding: 10px`，`border: 5px solid black`，那麼該元素的總寬度是多少？
  - A. 100px
  - B. 120px
  - C. 130px
  - D. 150px
3. 在 Flexbox 佈局中，哪個屬性用來設置元素之間的對齊方式？
  - A. `justify-content`
  - B. `align-items`
  - C. `flex-wrap`
  - D. `align-content`
4. 當 CSS 屬性 `box-sizing` 設為 `border-box` 時，元素的總寬度會包含以下哪一項？
  - A. 內容區域的寬度
  - B. 內邊距和邊框的尺寸
  - C. 內邊距和邊框的尺寸會被排除在外
  - D. 僅內容區域的寬度，邊框會扣除
5. Flexbox 佈局中，`flex-wrap: wrap` 的作用是？
  - A. 隱藏超出容器的元素
  - B. 強制所有元素在一行顯示

- C. 允許元素在容器寬度不足時自動換行
- D. 旋轉容器內容

#### 4.4 測驗題答案：

► 點擊展開看解答

#### 練習題

1. 使用 Flexbox 將三個 `.item` 元素放在一行，並使它們均勻分佈，且每個元素的寬度相等。
2. 使用 CSS Grid 創建一個三列的佈局，第一列的寬度為 `1fr`，第二列的寬度為 `2fr`，第三列的寬度為 `1fr`。並且給每個元素設定 `10px` 的間距。
3. 設計一個包含兩個區塊的頁面，使用 CSS 盒模型確保其中一個區塊有 `20px` 的內邊距，`5px` 的邊框，並且與另一個區塊保持 `10px` 的外邊距。

## 4.5 樣式與效果

:point\_right: [Demo: Typography](#) | [Demo: Decoration](#) | [Demo: Positioning](#) | [Demo: Dynamics](#)

:point\_right: [W3Schools - CSS Tutorial](#)

CSS（層疊樣式表）是用來控制 HTML 元素的外觀和排版的技術。透過 CSS，我們可以設計出美觀的網頁佈局、特效和動畫。以下是 CSS 中一些常用的樣式與效果，這些技巧將幫助你創建富有吸引力的網頁。

### 4.5.1 文字與版面

文字與版面屬性控制內容的可讀性與排列。

文字演進：

- 字體與大小：使用 `font-family`，`font-size`，`font-weight`。
- 排版優化：透過 `line-height` 與 `letter-spacing` 增加舒適度。
- \*\*自動換行 ( `word-wrap` )\*\*：控制長單字是否強制換行。
- \*\*多欄布局 ( `columns` )\*\*：將內容自動分成多欄（如同報紙）。

```
article {  
  columns: 2;  
  column-gap: 20px;  
  word-wrap: break-word;  
}
```

### 4.5.2 視覺裝飾

透過顏色、邊框與陰影提升介面的精緻感。

裝飾技巧：

- \*\*背景樣式 ( `background` )\*\*：包含 `background-color` 與可縮放的 `background-image`。

- **邊框 ( border )**：定義實線、虛線及圓角。
- **陰影效果 ( shadow )**： `box-shadow` 用於區塊， `text-shadow` 用於文字。

```
.card {  
  background-color: white;  
  border: 1px solid #ddd;  
  box-shadow: 2px 2px 10px rgba(0,0,0,0.1);  
}
```

#### [!TIP] 🚀 旅遊網站對照：裝飾與特效 (Effects)

在旅遊卡片中，圓角、陰影與過渡動畫是提升精緻感的關鍵。

傳統 CSS:

```
.travel-card {  
  border-radius: 1rem;          /* rounded-2xl */  
  box-shadow: 0 20px 25px -5px rgba(0,0,0,0.1); /* shadow-xl */  
  transition: box-shadow 0.3s;  
}  
.travel-card:hover {  
  box-shadow: 0 25px 50px -12px rgba(0,0,0,0.25); /* shadow-2xl */  
}
```

Tailwind CSS:

```
<div class="rounded-2xl shadow-xl hover:shadow-2xl transition-shadow">  
  <!-- 卡片內容 -->  
</div>
```

### 4.5.3 定位與流

控制元素在頁面中的物理位置與互動關係。

核心概念：

- **定位 ( position )**： `relative` (相對), `absolute` (絕對), `fixed` (固定)。
- **浮動 ( float )**：使元素浮動並讓文字環繞。
- **溢出處理 ( overflow )**：當內容太大時，使用 `hidden` 隱藏或 `scroll` 捲動。

### 4.5.4 動態效果與變形

增加與使用者的互動回饋。

動態效果：

- **變形 ( transform )**：旋轉 `rotate` , 縮放 `scale` , 平移 `translate` 。

- **過渡 ( transition )**：屬性變化的平滑補間動畫。
- **動畫 ( animation )**：搭配 **@keyframes** 定義複雜的循環動作。

```
button:hover {  
  transform: scale(1.1);  
  transition: transform 0.3s ease;  
}
```

---

## 測驗題

1. 以下哪個 CSS 屬性可以用來設定元素的背景顏色？
  - A. color
  - B. background-color
  - C. border-color
  - D. font-color
2. 使用 transition 時，若希望元素背景顏色在 1 秒內平滑變化，應使用以下哪個屬性？
  - A. transition-duration
  - B. transition-property
  - C. transition-timing-function
  - D. transition
3. 在 CSS 動畫中，@keyframes 用來設定：
  - A. 動畫的持續時間
  - B. 動畫的屬性
  - C. 動畫的起始和結束狀態
  - D. 動畫的方向
4. 如何給文本添加陰影？
  - A. box-shadow
  - B. text-shadow
  - C. shadow
  - D. color-shadow
5. 在 box-sizing: border-box 佈局中，width 屬性會包括：
  - A. 內容區域、內邊距和邊框
  - B. 內容區域
  - C. 內邊距和邊框
  - D. 內容區域和內邊距

## 4.5 測驗題答案：

- 點擊展開看解答

---

## 練習題

1. 設計一個 `div` 元素，當滑鼠懸停時，背景顏色從紅色變為綠色，並且這個過渡效果持續 0.7 秒。
2. 創建一個 3 欄的文本布局，每欄的寬度各自為 33%，並設置每欄之間的間距為 20px。
3. 使用 `transform` 屬性將一個 `div` 元素旋轉 45 度，並使它縮放至 1.2 倍大小。

## 4.6 其他

:point\_right: [W3Schools - CSS Flexbox](#)

### 4.6.1 假圖產生器

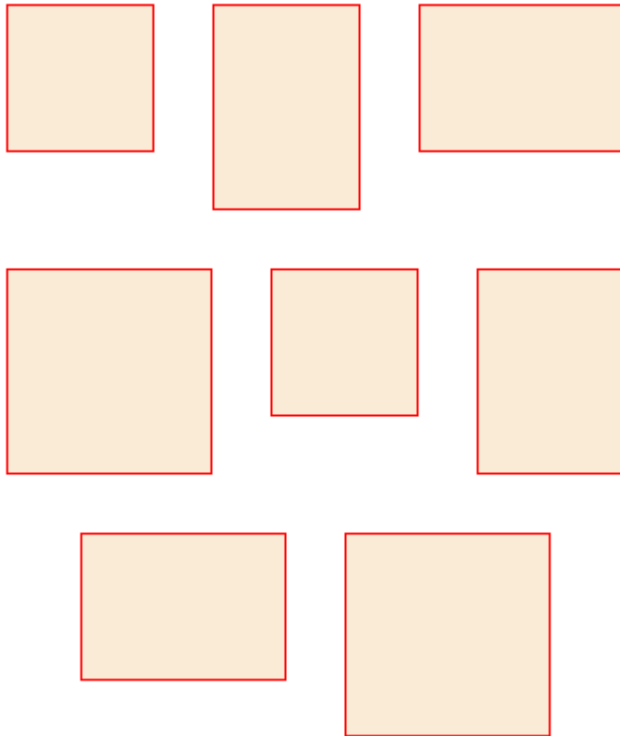
在使用 CSS 進行版面測試時，我們常需要一些假圖片來填充版位。**Picsum Photos** 是最受歡迎的工具。

- 基本用法：
  - 固定大小：`https://picsum.photos/200`
  - 指定寬高：`https://picsum.photos/300/200`
  - 隨機圖片：`https://picsum.photos/200?random=1`
- \*\*進階技巧 (補充)\*\*：
  - 灰階效果：`https://picsum.photos/200?grayscale` (適合復古或特定設計風格)
  - 模糊效果：`https://picsum.photos/200?blur=5` (模糊程度從 1-10)
  - 指定圖片 ID：`https://picsum.photos/id/237/200/300` (固定使用同一張圖)

### 4.6.2 假文字產生器

**Lorem Ipsum** 是一段沒有意義的拉丁文字，用於測試網頁的排版、字體對齊與段落樣式。

- **VS Code 手動輸入**：
    - `lorem`：按 Tab/Enter 會產生一段標準長度的文字。
    - `lorem10`：產生 10 個英文字。
  - **\*\*Emmet 進階應用 (補充)\*\***：
    - 產生多個段落：輸入 `p*3>lorem10` 並按 Tab。這會產生 3 個 `<p>` 標籤，每個標籤內含 10 個假文字。
  - **\*\*中文字產生器 (ctlorem)\*\***：
    - 需要安裝插件，輸入 `ctlorem50` 可產生 50 個中文字，這對於中文字型排版測試非常重要。
-



### 測驗題

1. 在 HTML 中使用 ``，圖片的寬度是多少？
  - A. 200px
  - B. 300px
  - C. 視窗寬度的 300%
  - D. 隨機
2. Lorem Ipsum 的主要用途是什麼？
  - A. 正規的網頁文案
  - B. 代碼註釋
  - C. 填充排版用的假文字
  - D. SEO 優化關鍵字
3. 如果想產生 50 個中文字，使用 `ctlorem` 插件應輸入什麼關鍵字？
  - A. `lorem50`
  - B. `ctlorem50`
  - C. `chineselorem50`
  - D. `text50`

### 4.6 測驗題答案：

► 點擊展開看解答

---

## 4.7 綜合練習：ShowMovie 影城訂票系統設計

請以「影城訂票系統」為主題，運用本章所學的 CSS 技術（選擇器、盒模型、佈局、裝飾、動態效果），完成以下三個進階練習。

### 1. 基礎挑戰：響應式電影海報牆

設計一個直觀的電影選單介面，讓使用者一眼就能看到正在熱映的作品。

- 佈局要求：
  - 建立一個寬度為 90% 的主容器，並在頁面中水平置中。
  - 使用 **Flexbox** 排版電影卡片，設定 `justify-content: space-around` 並允許換行 ( `flex-wrap: wrap` )。
- 卡片樣式：
  - 每張電影卡片要有固定寬度（例如 220px ）並設定圓角 `border-radius: 12px;`。
  - 海報圖片必須撐滿卡片寬度，且圖片下方要留白置入片名 ( `<h3>` ) 與分級資訊 ( `<span>` )。
  - 對片名使用 `margin: 10px 0;` 以確保視覺上的層次感。

### 2. 變形練習一：互動式視覺強化 (Hover & Shadow)

承接基礎挑戰，透過裝飾屬性與動態效果，提升使用者與介面的互動感。

- 陰影與層次：
  - 為每張卡片加上輕微的 `box-shadow`。
  - 當滑鼠移到卡片上時，陰影要變得更深、範圍更廣，模擬卡片「浮起來」的感覺。
- 變形與過渡：
  - 實作滑鼠懸停效果：讓卡片略微放大 5% ( `transform: scale(1.05);` )。
  - 加入 `transition: all 0.3s ease;` 確保放大的過程平滑流暢。
- 文字裝飾：
  - 點擊購票按鈕 ( `<a>` 標籤 ) 時，使用 `:active` 偽類讓按鈕產生被「按下」的視覺反饋。

### 3. 變形練習二：座位選取面板與偽類應用 (Grid & State)

設計一個模擬影廳座位選取的區塊，挑戰更精確的選擇器與佈局應用。

- 座位陣列：
  - 使用 CSS Grid 或 Flexbox 排列 8 排 x 10 列 的座位（可以是一個個的小 `div`）。
  - 每個座位寬高各 25px，並帶有圓角。
- 狀態選擇器：
  - 使用 `** :not(.taken) **`：選擇所有還沒被買走的座位，並設置懸停變色。
  - 使用 `** 屬性選擇器 [data-status="selected"] **`：標註當前選中的座位為綠色。
  - 使用 `** :nth-child()`：將影廳中間的黃金位置（例如第 4 與 第 5 列）標註為不同背景色。

- 浮動定位：
  - 設計一個「確認訂購」的區塊，使用 `position: sticky;` 將其固定在瀏覽器視窗底部，確保使用者無論滑到哪都能隨時提交。